UNIVERSIDADE PAULISTA – UNIP
CURSO DE ENGENHARIA DA COMPUTAÇÃO

WELLINGTON TOMIYOSHI SANTOS KOGA
RA: G773470

ESTAÇÃO DE MONITORAMENTO DE JARDIM INTELIGENTE:

simulação em VS Code, PlatformIO e Wokwi com ESP32

SÃO JOSÉ DOS CAMPOS
2026

WELLINGTON TOMIYOSHI SANTOS KOGA

ESTAÇÃO DE MONITORAMENTO DE JARDIM INTELIGENTE:

simulação em VS Code, PlatformIO e Wokwi com ESP32

Relatório técnico apresentado à disciplina Sistemas Digitais, do curso de Engenharia da Computação da Universidade Paulista – UNIP, como trabalho complementar da avaliação NP1.

Professor: Fabiano Cruz Nogueira

SÃO JOSÉ DOS CAMPOS

2026

RESUMO

Este relatório apresenta o desenvolvimento de uma estação de monitoramento de jardim inteligente baseada no microcontrolador ESP32, realizado como trabalho complementar da avaliação NP1 da disciplina Sistemas Digitais. O objetivo foi estruturar um ambiente profissional de desenvolvimento e simulação de sistemas embarcados e implementar um firmware capaz de monitorar o microclima e o solo. O projeto foi desenvolvido no Visual Studio Code, compilado com o PlatformIO e simulado no Wokwi. O sistema base lê a temperatura e a umidade do ar com um sensor DHT22 e a umidade do solo por meio de um potenciômetro, a cada dois segundos, exibe os dados em um display LCD 16x2 I2C e envia telemetria pela porta serial. Foram implementadas duas melhorias: a irrigação automática com relé e histerese, que liga abaixo de 30% e desliga acima de 40% de umidade do solo, e um sistema de alarme visual e sonoro com LEDs, buzzer e botão de silenciamento tratado por interrupção externa. O firmware utiliza temporização não bloqueante e ocupou 22,5% da memória flash e 6,7% da memória RAM. Os oito casos de teste realizados na simulação apresentaram o comportamento esperado. O projeto está disponível em repositório público no GitHub.

Palavras-chave: sistemas embarcados; ESP32; PlatformIO; Wokwi; automação.

SUMÁRIO

1 INTRODUÇÃO.....4
1.1 Objetivos.....4
2 CONFIGURAÇÃO DO AMBIENTE DE DESENVOLVIMENTO.....5
2.1 Etapa 1 – Instalação das extensões.....5
2.2 Etapa 2 – Licença do Wokwi.....5
2.3 Etapa 3 – Criação do projeto e regras de caminho.....6
2.4 Etapa 4 – Compilação e execução.....6
2.5 Versionamento no GitHub.....7
3 ARQUITETURA DE HARDWARE.....8
3.1 Justificativas técnicas.....8
4 DIAGRAMA DE BLOCOS.....10
5 ARQUIVOS DE CONFIGURAÇÃO.....12
5.1 platformio.ini.....12
5.2 wokwi.toml.....12
5.3 diagram.json.....13
6 FIRMWARE.....14
6.1 Organização do código.....14
6.2 Código-fonte comentado (src/main.cpp).....15
7 MELHORIAS IMPLEMENTADAS.....21
7.1 Melhoria 1 – Irrigação automática com atuador virtual.....21
7.2 Melhoria 2 – Alarme visual e sonoro para condições críticas.....21
8 TESTES E RESULTADOS.....23
8.1 Uso de memória.....25
8.2 Trecho do Monitor Serial.....25
9 CONCLUSÃO.....26
REFERÊNCIAS.....27

1 INTRODUÇÃO

A agricultura de precisão e a automação residencial dependem de sistemas embarcados capazes de medir o ambiente, tomar decisões e agir sem intervenção humana. Este trabalho apresenta o desenvolvimento de uma **Estação de Monitoramento de Jardim Inteligente** baseada no microcontrolador **ESP32**, capaz de medir continuamente a temperatura e a umidade do ar, estimar a umidade do solo, exibir os dados em um display LCD 16x2 I2C e enviar telemetria pela porta serial.

Todo o desenvolvimento foi feito em um ambiente profissional de simulação: o **Visual Studio Code** como editor, o **PlatformIO** como sistema de compilação e gerenciamento de bibliotecas e a extensão **Wokwi Simulator** para simular o hardware virtual. Além do projeto base, foram implementadas duas melhorias: um **sistema de irrigação automática** com relé e um **sistema de alarme visual e sonoro** para condições críticas, com botão de silenciar tratado por interrupção.

O trabalho segue o roteiro proposto por Nogueira (2026) e a documentação oficial das ferramentas e bibliotecas utilizadas (Adafruit, [20--]; Espressif Systems, [20--]; PlatformIO, [20--]; Wokwi, [20--]).

1.1 Objetivos

Os objetivos deste trabalho são:

a) configurar o ambiente VS Code + PlatformIO + Wokwi e compilar o firmware sem erros;
b) ler os sensores a cada 2 segundos, exibir os dados no LCD e enviar logs pela Serial a 115200 baud;
c) acionar automaticamente a irrigação quando o solo estiver seco;
d) alertar visualmente e sonoramente quando a temperatura passar de 35 °C ou a umidade do ar ficar abaixo de 20%.

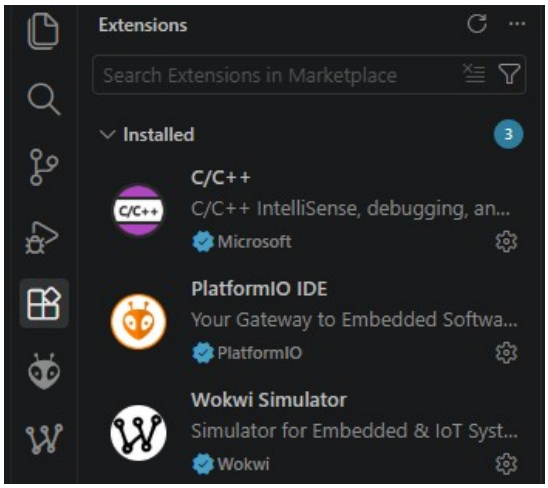
2 CONFIGURAÇÃO DO AMBIENTE DE DESENVOLVIMENTO

As etapas obrigatórias do roteiro foram executadas e documentadas conforme a seguir.

2.1 Etapa 1 – Instalação das extensões

No Marketplace do VS Code foram instaladas as extensões **PlatformIO IDE** (identificador platformio.platformio-ide) e **Wokwi Simulator** (identificador wokwi.wokwi-vscode). A Figura 1 mostra as extensões instaladas.

Figura 1 – Extensões PlatformIO IDE e Wokwi Simulator instaladas no VS Code

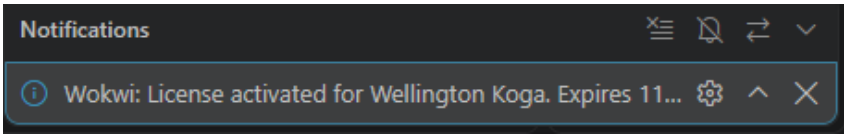


Fonte: Elaborado pelo autor (2026).

2.2 Etapa 2 – Licença do Wokwi

Pela Paleta de Comandos (F1 ou Ctrl+Shift+P) foi executado o comando **Wokwi: Request a New License Key**. O login foi feito no navegador, a licença gratuita foi autorizada e o VS Code confirmou a ativação. A Figura 2 mostra a confirmação exibida.

Figura 2 – Confirmação de ativação da licença do Wokwi no VS Code



Fonte: Elaborado pelo autor (2026).

2.3 Etapa 3 – Criação do projeto e regras de caminho

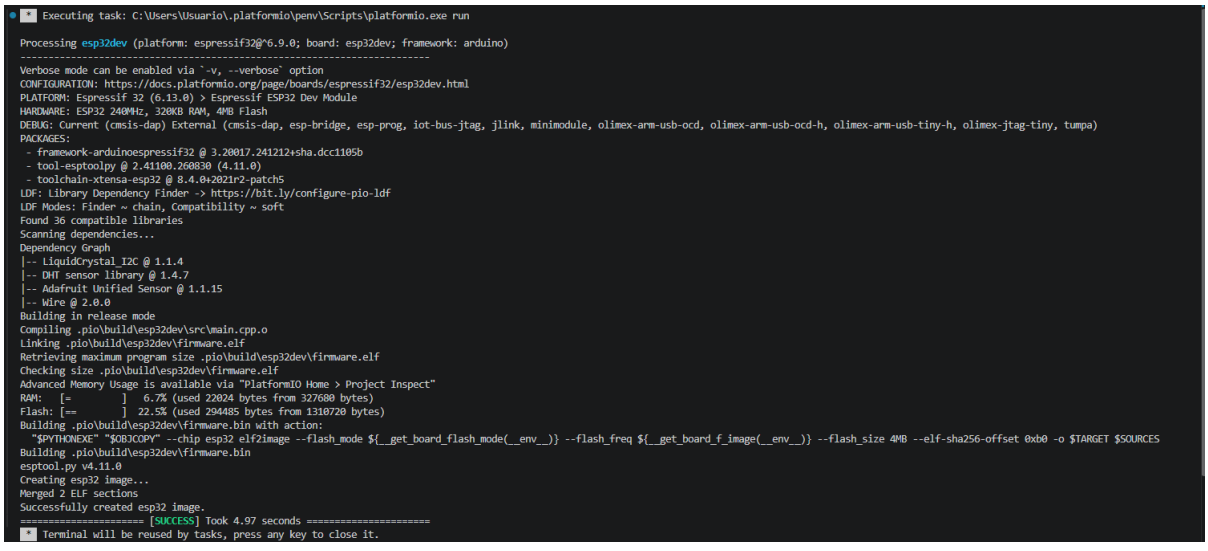
O projeto foi criado na pasta **B:\Projetos\jardim_inteligente**, um caminho sem espaços e sem acentos, respeitando a limitação de espaços em branco do sistema de compilação (CMake/Ninja). O nome do ambiente **[env:esp32dev]** do platformio.ini foi mantido igual ao caminho do firmware no wokwi.toml (.pio/build/**esp32dev**/firmware.bin), evitando o erro "Firmware not found".

2.4 Etapa 4 – Compilação e execução

A compilação foi feita pelo ícone ✔ **(Build)** do PlatformIO, com a confirmação **[SUCCESS]** no terminal (Figura 3). Em seguida, a simulação foi iniciada abrindo o arquivo diagram.json (ou pelo comando Wokwi: Start Simulator).

O ambiente utilizou a plataforma **Espressif 32 (6.13.0)** do PlatformIO, com o framework Arduino-ESP32 (pacote 3.20017, núcleo 2.0.17) e o toolchain xtensa-esp32 8.4.0. As bibliotecas LiquidCrystal_I2C 1.1.4, DHT sensor library 1.4.7 e Adafruit Unified Sensor 1.1.15 foram baixadas automaticamente a partir do lib_deps. A compilação levou 4,97 s.

Figura 3 – Compilação no PlatformIO concluída com [SUCCESS] e uso de RAM/Flash

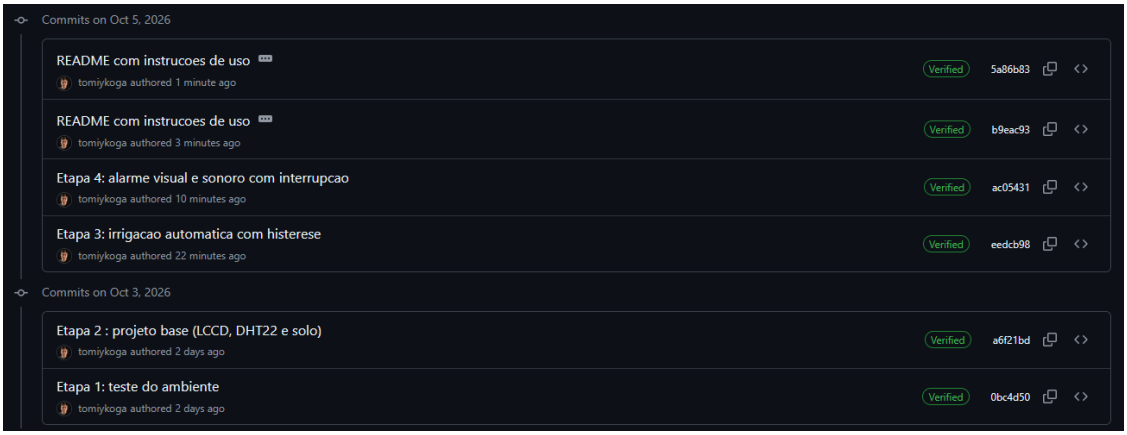


Fonte: Elaborado pelo autor (2026).

2.5 Versionamento no GitHub

O projeto foi publicado no repositório público **https://github.com/tomiykoga/jardim_inteligente**. Cada etapa do desenvolvimento gerou um commit próprio — teste do ambiente, projeto base, irrigação automática e sistema de alarme —, seguido do README com as instruções de uso. Assim, o histórico mostra a evolução incremental do firmware e do circuito. A Figura 4 apresenta esse histórico.

Figura 4 – Histórico de commits do repositório, uma etapa por commit



Fonte: Elaborado pelo autor (2026).

3 ARQUITETURA DE HARDWARE

O circuito virtual é composto pelos componentes do Quadro 1. Os pinos foram escolhidos considerando as particularidades do ESP32.

Quadro 1 – Componentes e ligações

Componente	Modelo no Wokwi	Pino(s) no ESP32	Função
Microcontrolador	ESP32 DevKit C v4	–	Processamento e controle de I/O
Display	LCD 16x2 com I2C (0x27)	SDA 21 / SCL 22, 5V, GND	Exibição local dos dados
Sensor de clima	DHT22	GPIO 4, 3V3, GND	Temperatura (°C) e umidade do ar (%)
Sensor de solo	Potenciômetro	GPIO 34 (ADC1), 3V3, GND	Simula umidade do solo 0–100%
Atuador (melhoria 1)	Módulo relé	GPIO 26, 5V, GND	Liga a bomba de irrigação
Indicador (melhoria 1)	LED azul + 220 Ω	GPIO 27	Irrigação ligada
Alarme (melhoria 2)	LED vermelho + 220 Ω	GPIO 32	Calor excessivo (> 35 °C)
Alarme (melhoria 2)	LED amarelo + 220 Ω	GPIO 33	Solo seco ou ar seco
Alarme (melhoria 2)	Buzzer	GPIO 18, GND	Alarme sonoro intermitente
Entrada (melhoria 2)	Botão	GPIO 14 (pull-up interno), GND	Silenciar alarme (interrupção)

Fonte: Elaborado pelo autor (2026).

3.1 Justificativas técnicas

As escolhas de componentes e de pinos foram feitas pelas seguintes razões:

a) **DHT22 no lugar do DHT11:** o DHT22 está disponível nativamente no Wokwi e tem maior faixa e precisão. Para o firmware, a troca é feita só na constante TIPO_DHT;

b) **GPIO 4 para o DHT22:** o roteiro sugere o pino 2, porém no ESP32 o GPIO 2 é um pino de strapping (influencia o modo de boot) e está ligado ao LED da placa. O GPIO 4 é um pino de uso geral sem essas restrições;

c) **GPIO 34 para o solo:** pertence ao ADC1. O ADC2 do ESP32 fica indisponível quando o Wi-Fi está ativo, então usar o ADC1 deixa o projeto pronto para uma futura expansão IoT. O ADC tem 12 bits (0 a 4095);

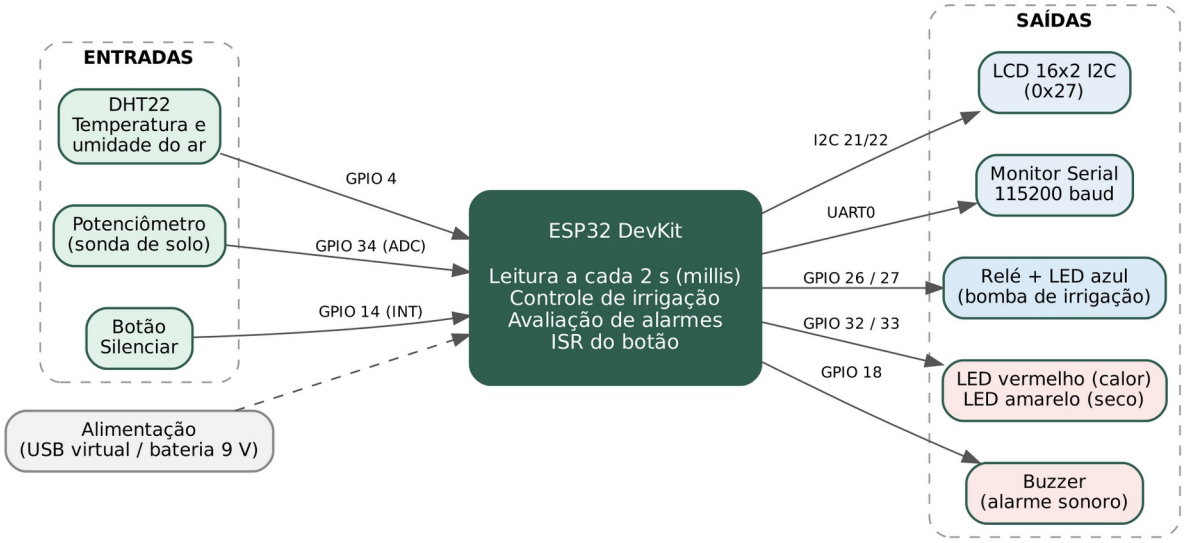
d) **I2C nos GPIO 21/22:** são os pinos I2C padrão do ESP32 no framework Arduino;

e) **alimentação:** o Wokwi alimenta a placa virtualmente e não possui um componente nativo de bateria 9 V. Em uma montagem física, a bateria seria ligada ao pino VIN/GND, de preferência por meio de um regulador de 5 V para não sobreaquecer o regulador da placa.

4 DIAGRAMA DE BLOCOS

A Figura 5 mostra o fluxo de informação do sistema: as entradas (sensores e botão) são processadas pelo ESP32, que comanda as saídas (display, serial, relé, LEDs e buzzer).

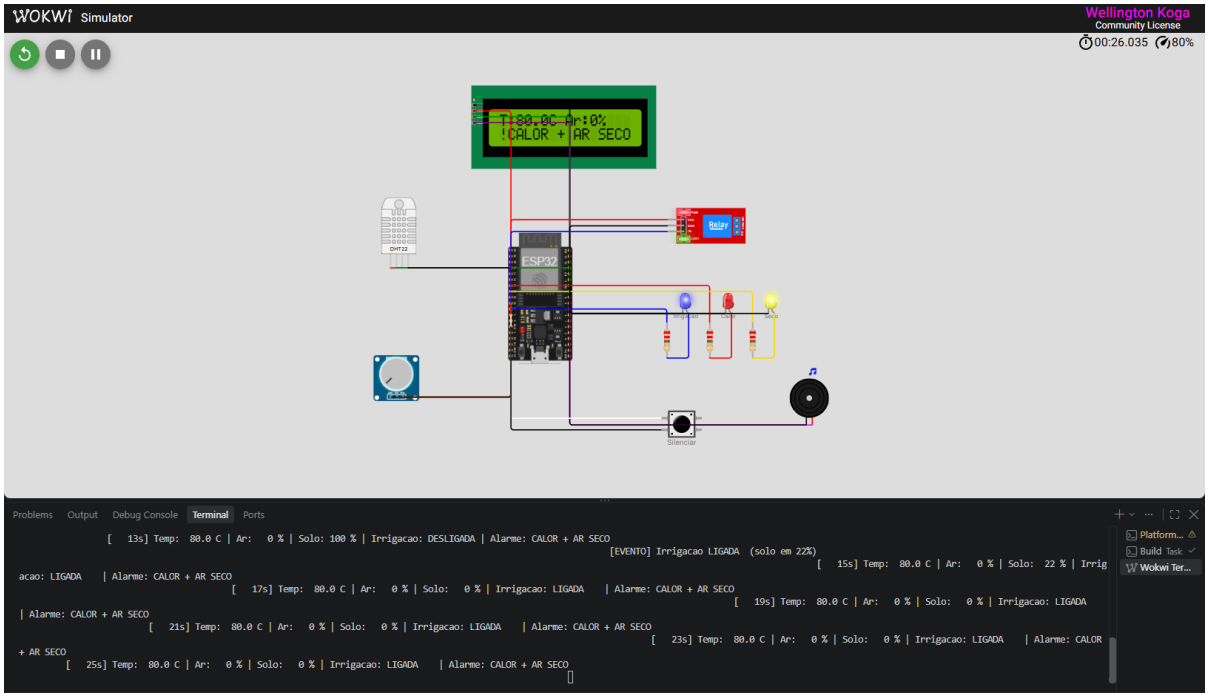
Figura 5 – Diagrama de blocos da estação



Fonte: Elaborado pelo autor (2026).

A Figura 6 mostra o circuito completo montado no Wokwi, com a simulação em execução.

Figura 6 – Circuito completo no Wokwi: alarme ativo com T = 80 °C e ar a 0% (LCD "!CALOR + AR SECO")



Fonte: Elaborado pelo autor (2026).

5 ARQUIVOS DE CONFIGURAÇÃO

5.1 platformio.ini

Define o ambiente de compilação **esp32dev**, a plataforma **espressif32**, o framework **Arduino**, a velocidade do monitor serial e as bibliotecas que o PlatformIO baixa automaticamente: LiquidCrystal_I2C (display), DHT sensor library e Adafruit Unified Sensor (dependência do DHT).

```
; ============================================================
;  Estação de Monitoramento de Jardim Inteligente
;  Arquivo de configuração do PlatformIO
;  Placa: ESP32 DevKit (framework Arduino)
; ============================================================

; O nome do ambiente [env:esp32dev] define a pasta de saída da
; compilação: .pio/build/esp32dev/  -> deve bater com o wokwi.toml
[env:esp32dev]
platform = espressif32@^6.9.0
board = esp32dev
framework = arduino

; Velocidade do Monitor Serial (mesma do Serial.begin no código)
monitor_speed = 115200

; Bibliotecas baixadas automaticamente pelo PlatformIO
lib_deps =
    marcoschwartz/LiquidCrystal_I2C@^1.1.4
    adafruit/DHT sensor library@^1.4.6
    adafruit/Adafruit Unified Sensor@^1.1.14
```

5.2 wokwi.toml

É a "ponte" entre o PlatformIO e o Wokwi: informa ao simulador onde estão os binários gerados pela compilação. O arquivo **.bin** é o firmware gravado na memória flash e o **.elf** contém os símbolos de depuração. O trecho **esp32dev** no caminho precisa ser idêntico ao nome do ambiente do platformio.ini.

```
# ============================================================
#  Ponte entre o PlatformIO e o simulador Wokwi
#  Informa ao Wokwi onde estão os binários gerados pelo Build.
#  "esp32dev" precisa ser IGUAL ao nome em [env:esp32dev]
#  do platformio.ini, senão aparece o erro "Firmware not found".
# ============================================================
[wokwi]
version = 1
firmware = '.pio/build/esp32dev/firmware.bin'
elf = '.pio/build/esp32dev/firmware.elf'
```

5.3 diagram.json

Descreve o circuito virtual em JSON. A lista **parts** declara cada componente (tipo, identificador, posição e atributos — por exemplo, o LCD com "pins": "i2c" e os resistores de 220 Ω). A lista **connections** declara cada fio no formato [origem, destino, cor, rota]. As duas primeiras conexões ligam o TX/RX do ESP32 ao Monitor Serial do Wokwi. Trecho representativo:

```
{ "type": "board-esp32-devkit-c-v4", "id": "esp", ... },
{ "type": "wokwi-lcd1602", "id": "lcd1", "attrs": { "pins": "i2c" } },
{ "type": "wokwi-dht22", "id": "dht1", ... },
{ "type": "wokwi-relay-module", "id": "relay1", ... },
...
[ "lcd1:SDA", "esp:21", "green", [] ],
[ "lcd1:SCL", "esp:22", "purple", [] ],
[ "dht1:SDA", "esp:4", "green", [] ],
[ "pot1:SIG", "esp:34", "orange", [] ],
[ "relay1:IN", "esp:26", "blue", [] ],
[ "btn1:1.l", "esp:14", "white", [] ],
```

O arquivo completo está no repositório do GitHub.

6 FIRMWARE

6.1 Organização do código

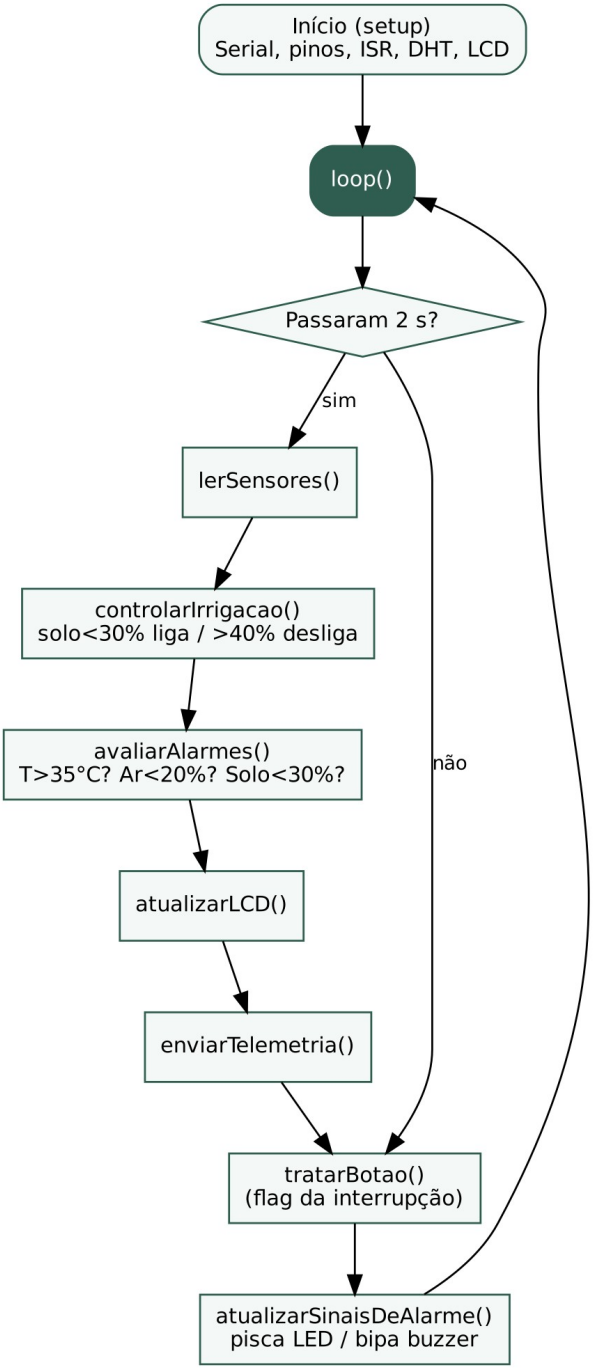
O firmware foi dividido em funções com uma responsabilidade cada, chamadas pelo loop(). A temporização é feita com **millis()** (não bloqueante): o loop roda continuamente e só executa o ciclo de leitura quando se passam 2 segundos. Assim, o botão, o pisca do LED e o bipe do buzzer continuam respondendo entre as leituras — algo que seria impossível usando delay(2000). O Quadro 2 resume a responsabilidade de cada função, e a Figura 7 apresenta o fluxograma do firmware.

Quadro 2 – Funções do firmware e suas responsabilidades

Função	**Responsabilidade**
lerSensores()	Lê o DHT22 e o ADC do solo, converte 0–4095 em 0–100% e detecta falha do sensor (NaN).
controlarIrrigacao()	Melhoria 1: liga/desliga relé e LED azul com histerese (30% / 40%).
avaliarAlarmes()	Melhoria 2: verifica T > 35 °C, ar < 20% e solo < 30%; acende o LED amarelo; rearma o alarme.
atualizarSinaisDeAlarme()	Pisca o LED vermelho e faz o buzzer bipar a cada 0,5 s sem delay.
isrBotaoSilenciar()	Rotina de interrupção: só sinaliza que o botão foi pressionado.
tratarBotao()	Trata o sinal da interrupção com debounce de 200 ms e silencia o buzzer.
atualizarLCD()	Linha 1: temperatura e umidade do ar. Linha 2: solo e irrigação, alternando com a mensagem de alerta.
enviarTelemetria()	Envia uma linha formatada com todos os dados pela Serial.

Fonte: Elaborado pelo autor (2026).

Figura 7 – Fluxograma do firmware



Fonte: Elaborado pelo autor (2026).

6.2 Código-fonte comentado (src/main.cpp)

```
/*
 * ============================================================================
 *  ESTAÇÃO DE MONITORAMENTO DE JARDIM INTELIGENTE
 *  Sistemas Digitais - Avaliação NP1 (Trabalho Complementar)
 *  Plataforma: ESP32 DevKit + VS Code + PlatformIO + Wokwi
 * ============================================================================
 *
 *  Projeto base:
 *    - DHT22  -> temperatura (°C) e umidade do ar (%)
```

```
 *    - Potenciômetro -> simula a sonda de umidade do solo (0% a 100%)
 *    - LCD 16x2 I2C -> exibição local dos dados
 *    - Serial 115200 baud -> telemetria / log
 *    - Leitura periódica a cada 2 segundos (sem usar delay)
 *
 *  Melhoria 1 - Irrigação automática:
 *    - Módulo relé (bomba d'água) + LED azul indicador
 *    - Liga quando o solo fica abaixo de 30% e desliga acima de 40%
 *      (histerese, para o relé não ficar ligando/desligando sem parar)
 *
 *  Melhoria 2 - Alarme visual e sonoro para condições críticas:
 *    - LED vermelho pisca se a temperatura passar de 35 °C
 *    - LED amarelo acende se o solo estiver seco (<30%) ou o ar seco (<20%)
 *    - Buzzer bipa de forma intermitente se temperatura > 35 °C
 *      ou umidade do ar < 20%
 *    - Botão "silenciar" ligado a uma INTERRUPÇÃO externa: desliga o som
 *      do buzzer até a condição crítica acabar (os LEDs continuam avisando)
 * ============================================================================
 */

#include <Arduino.h>
#include <Wire.h>               // comunicação I2C
#include <LiquidCrystal_I2C.h>  // display LCD com módulo I2C
#include <DHT.h>                // sensor de temperatura e umidade

// ----------------------------------------------------------------------------
//  MAPA DE PINOS (ESP32)
// ----------------------------------------------------------------------------
#define PINO_DHT          4    // Dado do DHT22
#define PINO_SOLO         34   // Potenciômetro (entrada analógica ADC1)
#define PINO_RELE         26   // Entrada IN do módulo relé (bomba)
#define PINO_LED_IRRIG    27   // LED azul: irrigação ligada
#define PINO_LED_CALOR    32   // LED vermelho: calor excessivo
#define PINO_LED_SECO     33   // LED amarelo: solo seco / ar seco
#define PINO_BUZZER       18   // Buzzer
#define PINO_BOTAO        14   // Botão para silenciar o alarme
// LCD I2C: SDA = GPIO 21 e SCL = GPIO 22 (pinos I2C padrão do ESP32)

// ----------------------------------------------------------------------------
//  PARÂMETROS DO SISTEMA
// ----------------------------------------------------------------------------
#define TIPO_DHT              DHT22
#define ENDERECO_LCD          0x27     // Endereço I2C do LCD no Wokwi
#define INTERVALO_LEITURA_MS  2000UL   // Leitura dos sensores a cada 2 s

#define SOLO_LIGA_IRRIGACAO      30    // Abaixo disso: liga a bomba (%)
#define SOLO_DESLIGA_IRRIGACAO   40    // Acima disso: desliga a bomba (%)
#define LIMITE_TEMP_CRITICA      35.0  // Calor excessivo (°C)
#define LIMITE_UMIDADE_AR_BAIXA  20.0  // Ar muito seco (%)

#define FREQ_BUZZER_HZ        2000     // Tom do alarme
#define PERIODO_PISCA_MS      500UL    // Pisca LED / bipa buzzer a cada 0,5 s
#define DEBOUNCE_BOTAO_MS     200UL    // Filtro contra trepidação do botão

// ----------------------------------------------------------------------------
//  OBJETOS DOS PERIFÉRICOS
// ----------------------------------------------------------------------------
LiquidCrystal_I2C lcd(ENDERECO_LCD, 16, 2);
DHT dht(PINO_DHT, TIPO_DHT);

// ----------------------------------------------------------------------------
//  VARIÁVEIS GLOBAIS DE ESTADO
// ----------------------------------------------------------------------------
float temperatura = 0;        // °C
float umidadeAr = 0;          // %
int   umidadeSolo = 0;        // %
bool  leituraDhtOk = false;   // false se o DHT22 falhar

bool irrigacaoLigada = false;
bool alarmeCalor = false;     // temperatura > 35 °C
bool alarmeArSeco = false;    // umidade do ar < 20%
bool alertaSoloSeco = false;  // solo < 30%
```

```
bool alarmeSilenciado = false;

bool mostrarTelaAlerta = false;  // alterna a linha 2 do LCD quando há alarme

unsigned long ultimaLeitura = 0;
unsigned long ultimoPisca = 0;
bool faseAlarme = false;         // liga/desliga do pisca e do bipe
bool buzzerTocando = false;      // estado atual do buzzer

// Variáveis usadas dentro da interrupção precisam ser "volatile"
volatile bool botaoFoiPressionado = false;
unsigned long ultimoBotao = 0;

// ---------------------------------------------------------------------------
//  ROTINA DE INTERRUPÇÃO (ISR) DO BOTÃO
//  Deve ser curta: apenas sinaliza o evento; o tratamento fica no loop().
//  IRAM_ATTR coloca a função na RAM interna, exigência do ESP32 para ISRs.
// ---------------------------------------------------------------------------
void IRAM_ATTR isrBotaoSilenciar() {
  botaoFoiPressionado = true;
}

// ---------------------------------------------------------------------------
//  LEITURA DOS SENSORES
// ---------------------------------------------------------------------------
void lerSensores() {
  // DHT22: temperatura e umidade do ar
  float t = dht.readTemperature();
  float h = dht.readHumidity();

  if (isnan(t) || isnan(h)) {
    leituraDhtOk = false;            // falha na comunicação com o sensor
  } else {
    leituraDhtOk = true;
    temperatura = t;
    umidadeAr = h;
  }

  // Solo: o ADC do ESP32 tem 12 bits (0 a 4095).
  // Convertemos para porcentagem de 0% (seco) a 100% (encharcado).
  int leituraBruta = analogRead(PINO_SOLO);
  umidadeSolo = map(leituraBruta, 0, 4095, 0, 100);
  umidadeSolo = constrain(umidadeSolo, 0, 100);
}

// ---------------------------------------------------------------------------
//  MELHORIA 1: CONTROLE DA IRRIGAÇÃO COM HISTERESE
// ---------------------------------------------------------------------------
void controlarIrrigacao() {
  if (!irrigacaoLigada && umidadeSolo < SOLO_LIGA_IRRIGACAO) {
    irrigacaoLigada = true;
    Serial.printf("[EVENTO] Irrigacao LIGADA  (solo em %d%%)\n", umidadeSolo);
  } else if (irrigacaoLigada && umidadeSolo > SOLO_DESLIGA_IRRIGACAO) {
    irrigacaoLigada = false;
    Serial.printf("[EVENTO] Irrigacao DESLIGADA (solo em %d%%)\n", umidadeSolo);
  }

  digitalWrite(PINO_RELE, irrigacaoLigada ? HIGH : LOW);
  digitalWrite(PINO_LED_IRRIG, irrigacaoLigada ? HIGH : LOW);
}

// ---------------------------------------------------------------------------
//  MELHORIA 2: AVALIAÇÃO DAS CONDIÇÕES CRÍTICAS
// ---------------------------------------------------------------------------
void avaliarAlarmes() {
  bool calorAntes = alarmeCalor;
  bool arSecoAntes = alarmeArSeco;

  // Só avalia temperatura/umidade do ar se o DHT22 respondeu
  alarmeCalor  = leituraDhtOk && (temperatura > LIMITE_TEMP_CRITICA);
  alarmeArSeco = leituraDhtOk && (umidadeAr < LIMITE_UMIDADE_AR_BAIXA);
  alertaSoloSeco = (umidadeSolo < SOLO_LIGA_IRRIGACAO);
```

```
  // Registra no Serial quando um alarme começa
  if (alarmeCalor && !calorAntes) {
    Serial.printf("[ALERTA] Temperatura critica: %.1f C\n", temperatura);
  }
  if (alarmeArSeco && !arSecoAntes) {
    Serial.printf("[ALERTA] Ar muito seco: %.0f%%\n", umidadeAr);
  }

  // Quando não há mais condição crítica, o alarme é "rearmado":
  // na próxima ocorrência o buzzer volta a tocar.
  if (!alarmeCalor && !alarmeArSeco && alarmeSilenciado) {
    alarmeSilenciado = false;
    Serial.println("[EVENTO] Condicoes normalizadas - alarme rearmado");
  }

  // LED amarelo: aceso fixo enquanto o solo ou o ar estiverem secos
  digitalWrite(PINO_LED_SECO, (alertaSoloSeco || alarmeArSeco) ? HIGH : LOW);
}

// ---------------------------------------------------------------------------
//  Pisca o LED vermelho e faz o buzzer bipar SEM usar delay(),
//  assim o resto do programa continua rodando normalmente.
// ---------------------------------------------------------------------------
void atualizarSinaisDeAlarme() {
  unsigned long agora = millis();
  if (agora - ultimoPisca >= PERIODO_PISCA_MS) {
    ultimoPisca = agora;
    faseAlarme = !faseAlarme;
  }

  // LED vermelho pisca enquanto houver calor excessivo
  digitalWrite(PINO_LED_CALOR, (alarmeCalor && faseAlarme) ? HIGH : LOW);

  // Buzzer bipa se houver condição crítica e o alarme não foi silenciado
  // (só chama tone/noTone quando o estado muda, para não sobrecarregar)
  bool deveTocar = (alarmeCalor || alarmeArSeco) && !alarmeSilenciado && faseAlarme;
  if (deveTocar && !buzzerTocando) {
    tone(PINO_BUZZER, FREQ_BUZZER_HZ);
    buzzerTocando = true;
  } else if (!deveTocar && buzzerTocando) {
    noTone(PINO_BUZZER);
    buzzerTocando = false;
  }
}

// ---------------------------------------------------------------------------
//  Trata o botão (sinalizado pela interrupção) com debounce por software
// ---------------------------------------------------------------------------
void tratarBotao() {
  if (!botaoFoiPressionado) return;
  botaoFoiPressionado = false;

  unsigned long agora = millis();
  if (agora - ultimoBotao < DEBOUNCE_BOTAO_MS) return;  // trepidação: ignora
  ultimoBotao = agora;

  if (alarmeCalor || alarmeArSeco) {
    alarmeSilenciado = true;
    noTone(PINO_BUZZER);
    buzzerTocando = false;
    Serial.println("[EVENTO] Alarme sonoro SILENCIADO pelo botao");
  }
}

// ---------------------------------------------------------------------------
//  Escreve uma linha completa no LCD (completa com espaços até 16 colunas,
//  evitando "sobras" de textos anteriores sem precisar de lcd.clear()).
// ---------------------------------------------------------------------------
void escreverLinhaLCD(uint8_t linha, const char *texto) {
  char buffer[17];
  snprintf(buffer, sizeof(buffer), "%-16s", texto);
```

```
  lcd.setCursor(0, linha);
  lcd.print(buffer);
}

// ---------------------------------------------------------------------------
//  ATUALIZAÇÃO DO DISPLAY LCD 16x2
//  Linha 1: temperatura e umidade do ar
//  Linha 2: umidade do solo e estado da irrigação
//           (se houver alarme, alterna com a mensagem de alerta)
// ---------------------------------------------------------------------------
void atualizarLCD() {
  char linha[17];

  // Linha 1
  if (leituraDhtOk) {
    snprintf(linha, sizeof(linha), "T:%.1fC Ar:%.0f%%", temperatura, umidadeAr);
  } else {
    snprintf(linha, sizeof(linha), "Erro no DHT22!");
  }
  escreverLinhaLCD(0, linha);

  // Linha 2
  bool haAlarme = alarmeCalor || alarmeArSeco;
  if (haAlarme && mostrarTelaAlerta) {
    if (alarmeCalor && alarmeArSeco) {
      snprintf(linha, sizeof(linha), "!CALOR + AR SECO");
    } else if (alarmeCalor) {
      snprintf(linha, sizeof(linha), "!! CALOR >35C !!");
    } else {
      snprintf(linha, sizeof(linha), "!AR SECO (<20%%)!");
    }
  } else {
    snprintf(linha, sizeof(linha), "Solo%3d%% Irr:%s",
             umidadeSolo, irrigacaoLigada ? "ON" : "OFF");
  }
  escreverLinhaLCD(1, linha);

  // Na próxima atualização mostra a outra tela
  mostrarTelaAlerta = haAlarme ? !mostrarTelaAlerta : false;
}

// ---------------------------------------------------------------------------
//  TELEMETRIA VIA SERIAL (115200 baud)
// ---------------------------------------------------------------------------
void enviarTelemetria() {
  unsigned long segundos = millis() / 1000;

  if (leituraDhtOk) {
    Serial.printf("[%5lus] Temp: %5.1f C | Ar: %3.0f %% | ",
                  segundos, temperatura, umidadeAr);
  } else {
    Serial.printf("[%5lus] Temp: ERRO    | Ar: ERRO  | ", segundos);
  }

  const char *estado = "NORMAL";
  if (alarmeCalor && alarmeArSeco) estado = "CALOR + AR SECO";
  else if (alarmeCalor)            estado = "CALOR";
  else if (alarmeArSeco)           estado = "AR SECO";

  Serial.printf("Solo: %3d %% | Irrigacao: %-9s | Alarme: %s%s\n",
                umidadeSolo,
                irrigacaoLigada ? "LIGADA" : "DESLIGADA",
                estado,
                alarmeSilenciado ? " (silenciado)" : "");
}

// ---------------------------------------------------------------------------
//  SETUP: executa uma vez ao ligar a placa
// ---------------------------------------------------------------------------
void setup() {
  Serial.begin(115200);
  Serial.println();
```

```
  Serial.println("=== Estacao de Monitoramento de Jardim Inteligente ===");

  // Saídas digitais
  pinMode(PINO_RELE, OUTPUT);
  pinMode(PINO_LED_IRRIG, OUTPUT);
  pinMode(PINO_LED_CALOR, OUTPUT);
  pinMode(PINO_LED_SECO, OUTPUT);
  pinMode(PINO_BUZZER, OUTPUT);

  // Botão com resistor de pull-up interno: solto = HIGH, pressionado = LOW
  pinMode(PINO_BOTAO, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(PINO_BOTAO), isrBotaoSilenciar, FALLING);

  // Entrada analógica do solo (resolução de 12 bits)
  analogReadResolution(12);

  // Sensores e display
  dht.begin();
  Wire.begin(21, 22);  // SDA, SCL
  lcd.init();
  lcd.backlight();

  // Tela de abertura + teste rápido dos LEDs
  escreverLinhaLCD(0, "Jardim Intelig.");
  escreverLinhaLCD(1, "Iniciando...");
  digitalWrite(PINO_LED_IRRIG, HIGH);
  digitalWrite(PINO_LED_CALOR, HIGH);
  digitalWrite(PINO_LED_SECO, HIGH);
  delay(1500);  // único delay do programa: só na inicialização
  digitalWrite(PINO_LED_IRRIG, LOW);
  digitalWrite(PINO_LED_CALOR, LOW);
  digitalWrite(PINO_LED_SECO, LOW);

  Serial.println("Sistema pronto. Leituras a cada 2 segundos.");

  // Força a primeira leitura logo no início do loop
  ultimaLeitura = millis() - INTERVALO_LEITURA_MS;
}

// ------------------------------------------------------------------------
//  LOOP: executa continuamente
//  Usa millis() para temporização não bloqueante: o botão, o pisca do LED
//  e o bipe do buzzer continuam respondendo entre uma leitura e outra.
// ------------------------------------------------------------------------
void loop() {
  unsigned long agora = millis();

  if (agora - ultimaLeitura >= INTERVALO_LEITURA_MS) {
    ultimaLeitura = agora;

    lerSensores();          // 1. Lê DHT22 e solo
    controlarIrrigacao();   // 2. Melhoria 1: relé + LED azul
    avaliarAlarmes();       // 3. Melhoria 2: verifica condições críticas
    atualizarLCD();         // 4. Mostra no display
    enviarTelemetria();     // 5. Envia log pela Serial
  }

  tratarBotao();              // Botão de silenciar (via interrupção)
  atualizarSinaisDeAlarme();  // Pisca LED vermelho e bipa o buzzer
}
```

7 MELHORIAS IMPLEMENTADAS

7.1 Melhoria 1 – Irrigação automática com atuador virtual

Foi adicionado um **módulo relé** no GPIO 26, representando o acionamento de uma bomba d'água, e um **LED azul** no GPIO 27 como indicador. A lógica liga a irrigação quando a umidade do solo cai **abaixo de 30%** e só desliga quando ela passa de **40%**.

Essa diferença entre o ponto de ligar e o de desligar é chamada de **histerese**. Sem ela, com o solo oscilando perto de 30% (29%, 30%, 29%...), o relé ficaria ligando e desligando a cada leitura, o que desgasta o relé e a bomba. Com a faixa de 10%, a bomba só desliga depois que o solo estiver realmente úmido. O estado aparece no LCD (Irr:ON / Irr:OFF) e cada mudança gera um evento na Serial.

7.2 Melhoria 2 – Alarme visual e sonoro para condições críticas

O sistema de alarme é formado pelos seguintes elementos:

a) **LED vermelho (GPIO 32):** pisca enquanto a temperatura estiver acima de 35 °C;
b) **LED amarelo (GPIO 33):** fica aceso com solo seco (< 30%) ou ar seco (< 20%);
c) **buzzer (GPIO 18):** bipa de forma intermitente (2 kHz, 0,5 s ligado / 0,5 s desligado) quando a temperatura passa de 35 °C ou a umidade do ar fica abaixo de 20%;
d) **LCD:** a segunda linha alterna entre os dados do solo e a mensagem de alerta (por exemplo, "!! CALOR >35C !!");
e) **botão Silenciar (GPIO 14):** configurado com pull-up interno e ligado a uma **interrupção externa** na borda de descida (FALLING). A ISR é curta e marcada com IRAM_ATTR (exigência do ESP32); ela apenas levanta uma flag volatile, e o tratamento com debounce é feito no loop. O botão desliga só o som — os LEDs continuam avisando — e o alarme é rearmado automaticamente quando as condições voltam ao normal.

O pisca do LED e o bipe do buzzer também usam millis(), e as funções tone()/noTone() só são chamadas quando o estado do buzzer muda, evitando processamento desnecessário.

8 TESTES E RESULTADOS

O firmware foi compilado sem erros nem avisos. Os testes foram realizados alterando os valores dos sensores durante a simulação (clicando no DHT22 para ajustar temperatura/umidade e girando o potenciômetro). O Quadro 3 lista os casos de teste e os resultados obtidos.

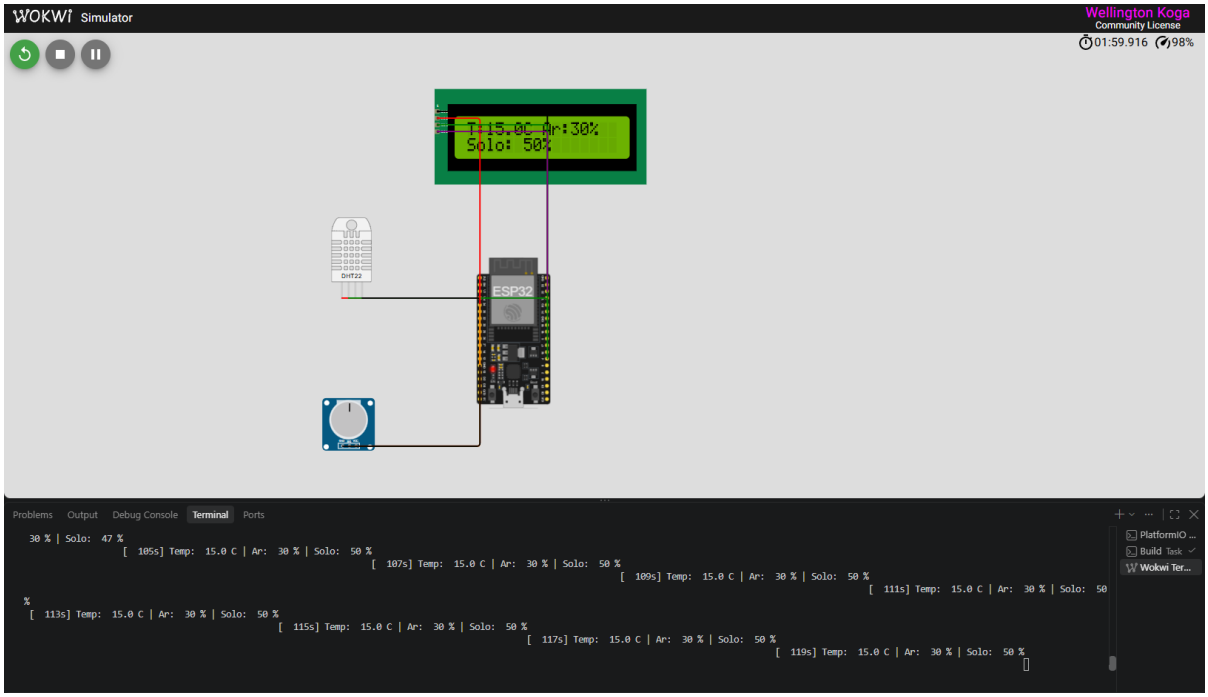
Quadro 3 – Casos de teste

#	Ação na simulação	Resultado esperado	Obtido
1	Iniciar a simulação (T = 25 °C, ar = 55%, solo = 50%)	Tela de abertura; depois dados no LCD e logs a cada 2 s; alarme NORMAL	OK
2	Girar o potenciômetro para menos de 30%	Relé e LED azul ligam; LED amarelo acende; LCD mostra Irr:ON	OK
3	Subir o solo para 35% (entre 30% e 40%)	Irrigação continua ligada (histerese)	OK
4	Subir o solo para mais de 40%	Relé e LED azul desligam; LCD mostra Irr:OFF	OK
5	Ajustar a temperatura do DHT22 para 38 °C	LED vermelho pisca; buzzer bipa; LCD alterna com "!! CALOR >35C !!"	OK
6	Apertar o botão Silenciar	Buzzer para; LED vermelho continua; evento na Serial	OK
7	Voltar a temperatura para 25 °C	Alarmes apagam; Serial mostra "alarme rearmado"	OK
8	Ajustar a umidade do ar para 15%	LED amarelo acende; buzzer bipa; LCD alterna com "!AR SECO (<20%)!"	OK

Fonte: Elaborado pelo autor (2026).

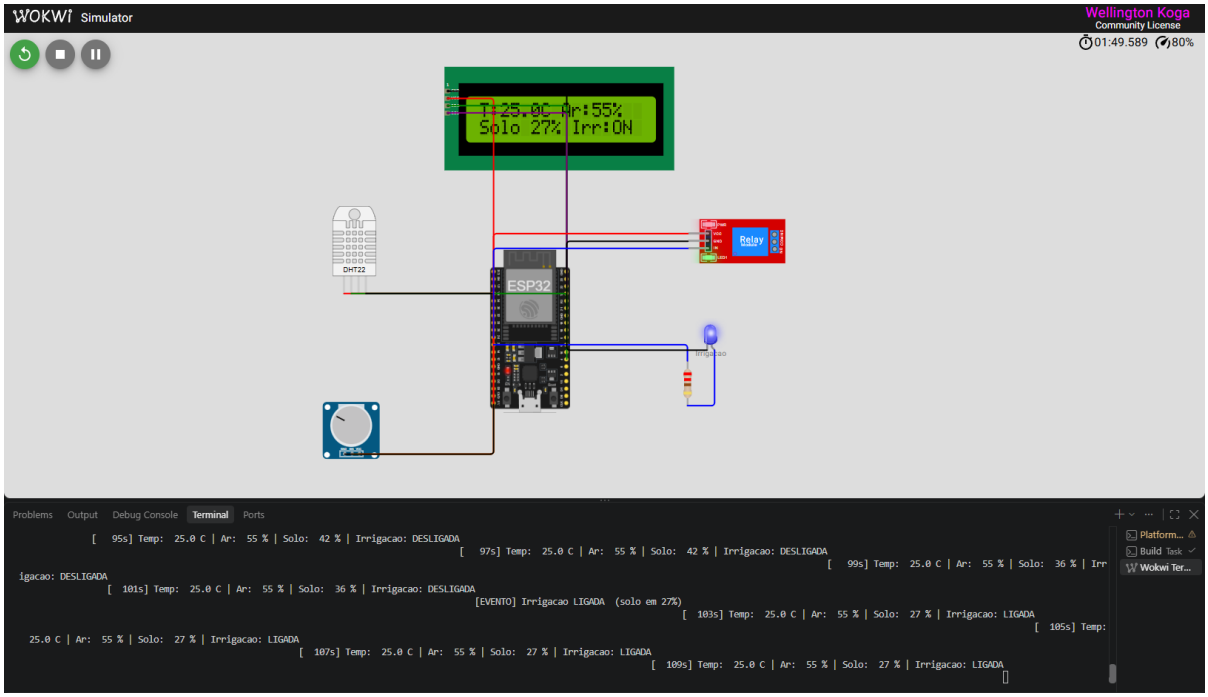
Todos os casos de teste apresentaram o comportamento esperado. As Figuras 8 e 9 mostram o projeto base e a irrigação automática em funcionamento; o alarme com calor e ar seco aparece na Figura 6. Em todas elas o Monitor Serial, na parte de baixo, registra as leituras a cada 2 segundos e os eventos.

Figura 8 – Projeto base (Etapa 2): LCD com temperatura, umidade do ar e do solo; Serial a cada 2 s



Fonte: Elaborado pelo autor (2026).

Figura 9 – Irrigação automática: solo em 27%, relé e LED azul acionados, LCD "Irr:ON" e evento na Serial



Fonte: Elaborado pelo autor (2026).

8.1 Uso de memória

Na compilação final (Figura 3), o firmware ocupou **294.485 bytes de Flash (22,5% de 1.310.720 bytes)** e **22.024 bytes de RAM (6,7% de 327.680 bytes)**. Há bastante espaço livre para futuras expansões, como Wi-Fi e MQTT.

8.2 Trecho do Monitor Serial

Trecho registrado durante o teste da Figura 6 (temperatura em 80 °C, umidade do ar em 0% e o potenciômetro sendo girado):

```
[   13s] Temp:  80.0 C | Ar:   0 % | Solo: 100 % | Irrigacao: DESLIGADA | Alarme: CALOR + AR SECO
[EVENTO] Irrigacao LIGADA  (solo em 22%)
[   15s] Temp:  80.0 C | Ar:   0 % | Solo:  22 % | Irrigacao: LIGADA    | Alarme: CALOR + AR SECO
[   17s] Temp:  80.0 C | Ar:   0 % | Solo:   0 % | Irrigacao: LIGADA    | Alarme: CALOR + AR SECO
```

9 CONCLUSÃO

O trabalho permitiu montar uma bancada virtual completa de desenvolvimento embarcado com VS Code, PlatformIO e Wokwi, passando pela configuração do ambiente, pela compilação e pela simulação de um sistema real de monitoramento. O projeto base cumpriu os requisitos de leitura periódica dos sensores, exibição no LCD I2C e telemetria serial.

As duas melhorias deixaram a estação mais próxima de um produto real: a irrigação automática com histerese evita o acionamento repetitivo do atuador, e o sistema de alarmes mostra na prática o uso de interrupções externas, debounce por software e temporização não bloqueante com millis(). Como trabalhos futuros, a estação pode ganhar conectividade Wi-Fi com envio de dados via MQTT, um sensor de luminosidade (LDR) para modos dia/noite e o ajuste dos limites pela Serial.

REFERÊNCIAS

ADAFRUIT. **DHT sensor library**. [S. l.]: GitHub, [20--]. Disponível em: https://github.com/adafruit/DHT-sensor-library. Acesso em: 5 out. 2026.

ESPRESSIF SYSTEMS. **ESP32 technical reference manual**. Shanghai: Espressif Systems, [20--]. Disponível em: https://www.espressif.com. Acesso em: 5 out. 2026.

NOGUEIRA, Fabiano Cruz. **Desafio prático**: estação de monitoramento de jardim inteligente. São José dos Campos: Universidade Paulista, 2026. Roteiro de atividade da disciplina Sistemas Digitais.

PLATFORMIO. **PlatformIO documentation**. [S. l.]: PlatformIO Labs, [20--]. Disponível em: https://docs.platformio.org. Acesso em: 5 out. 2026.

WOKWI. **Wokwi simulator documentation**. [S. l.]: Wokwi, [20--]. Disponível em: https://docs.wokwi.com. Acesso em: 5 out. 2026.